

Using Language Models

How to use n-gram LMs?

- Computing probability

$$P(\text{high school } \mathbf{principal}) > P(\text{high school } \mathbf{principle})$$

- Completion

$$\arg \max_{w \in V} P(w | \text{where, is, SFU})$$

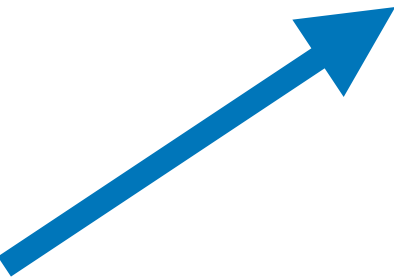
- Generating text

Computing the probability of a sentence

Apply the Chain Rule: the trigram model

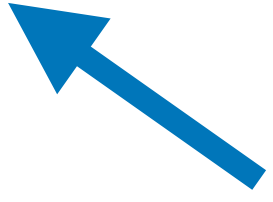
$$\begin{aligned} P(w_1, \dots, w_n) \\ &\approx P(w_1)P(w_2 \mid w_1)P(w_3 \mid w_1, w_2) \dots P(w_n \mid w_{n-2}, w_{n-1}) \\ &\approx P(w_1)P(w_2 \mid w_1) \prod_{i=3}^n P(w_i \mid w_{i-2}, w_{i-1}) \end{aligned}$$

Not trigrams!
Pad our sentence
with <s>
(start of sentence
markers)



$$P(w_1, \dots, w_n) \approx \prod_{i=1}^n P(w_i \mid w_{i-2}, w_{i-1})$$

Not proper
distribution unless we
add a **stop** symbol
(or </s> end of
sentence marker)



$$P(w_1) = P(w_1 \mid \langle s \rangle \langle s \rangle)$$

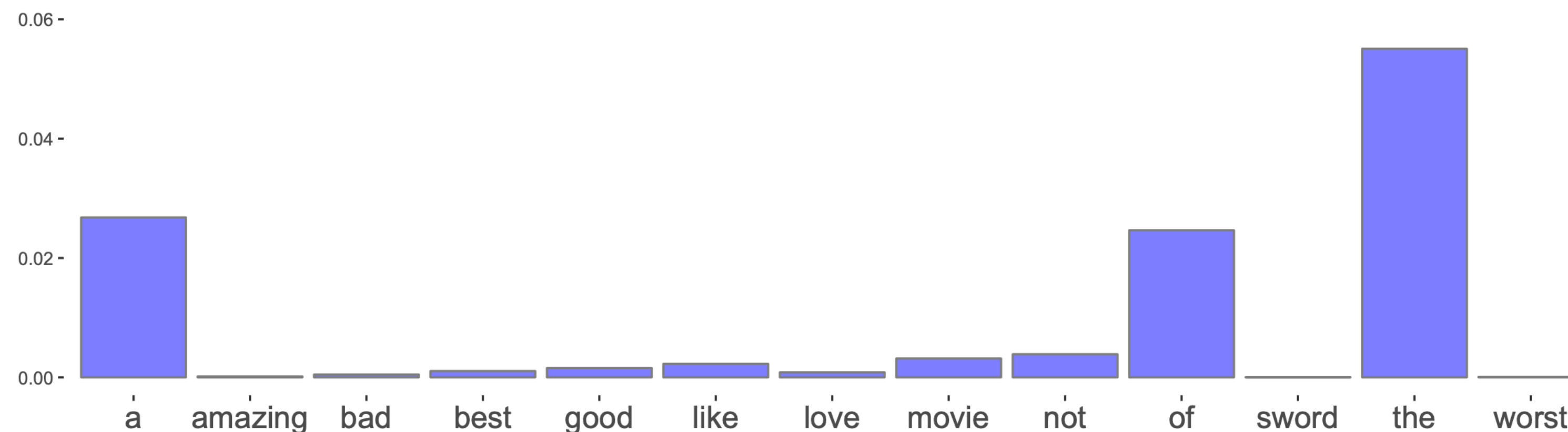
$$P(w_2 \mid w_1) = P(w_2 \mid \langle s \rangle w_1)$$

Generating text from n-grams

generative model

- How do you generate text from an n-gram model?
- Sample from distribution, generating tokens from left to right
- Use the last $n - 1$ words for context
- Select from multinomial over the vocabulary that include a **STOP** token. Repeat until **STOP** is generated.

context1	context2	generated word
START	START	The
START	The	dog
The	dog	walked
dog	walked	in



Example generation: Shakespeare

1
gram

–To him swallowed confess hear both. Which. Of save on trail for are ay device and rote life have

–Hill he late speaks; or! a more to leg less first you enter

2
gram

–Why dost stand forth thy canopy, forsooth; he is this palpable hit the King Henry. Live king. Follow.

–What means, sir. I confess she? then all sorts, he is trim, captain.

3
gram

–Fly, and will rid me these news of price. Therefore the sadness of parting, as they say, 'tis done.

–This shall forbid it should be branded, if renown made it empty.

4
gram

–King Henry. What! I will go seek the traitor Gloucester. Exeunt some of the watch. A great banquet serv'd in;

–It cannot be but so.

<https://web.stanford.edu/~jurafsky/slp3/3.pdf>

Typical LMs are not sufficient to handle long-range dependencies

“**Alice/Bob** could not go to work that day because
she/he had a doctor’s appointment”

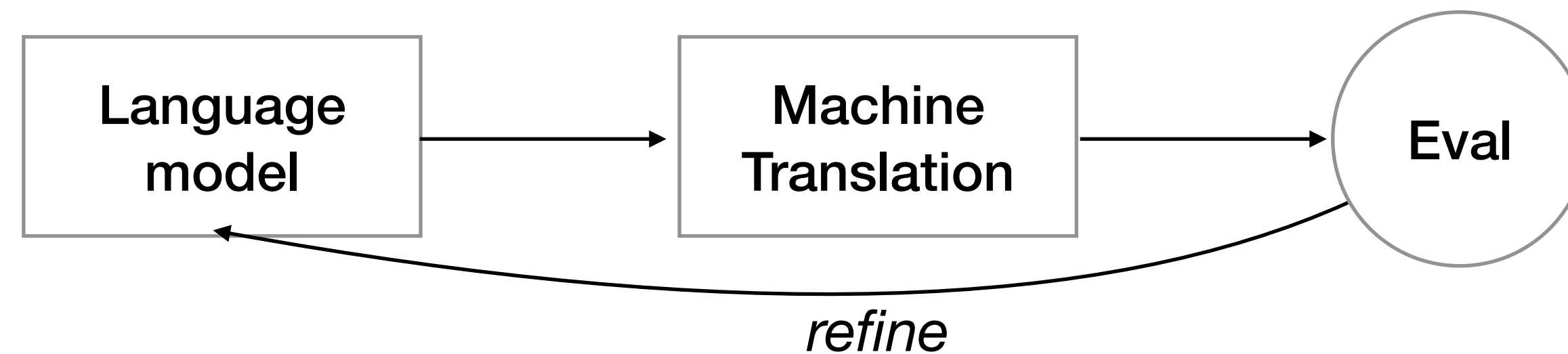
Evaluating Language Models

Evaluation

- **Extrinsic**: measure how useful the language model is at some task (MT, ASR, etc).
- **Intrinsic**: measure how good we are at modeling language

Extrinsic evaluation

- Train LM -> apply to task -> observe accuracy



- Directly optimized for downstream tasks
 - higher task accuracy -> better model
- Expensive, time consuming
- Hard to optimize downstream objective (indirect feedback)

Evaluating language models

- A good language model should assign higher probability to typical, grammatically correct sentences
- Research process:
 - **Train** parameters on a suitable training corpus
 - Assumption: observed sentences $\sim$ good sentences
 - **Test** on *different, unseen* corpus
 - Training on any part of test set not acceptable!
 - **Evaluation metric**



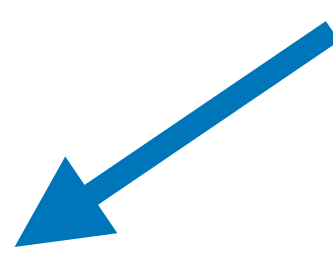
Perplexity

- Measure of how well a probability distribution (or model) predicts a sample

- For a corpus T with sentences $s_1, s_2, \dots, s_m$

$$\text{ppl}(T) = 2^{-\ell} \text{ where } -\ell = -\frac{1}{M} \sum_{i=1}^m \log_2 P(s_i)$$

cross entropy
between the
empirical
distribution of test
corpus and the
language model



where M is the total number of words in test corpus

- Unigram model: $-\ell = -\frac{1}{M} \sum_{i=1}^m \sum_{j=1}^{n_i} \log_2 P(w_j^{(i)}) = -\sum_{w \in V} \frac{C(w)}{M} \log_2 P(w)$
- Minimizing perplexity ~ maximizing probability

Intuition: Measure of model's uncertainty about next word
branching factor

Pros and cons of perplexity

Pros	Cons
Easy to compute	Domain match between train and test
standardized	Limited to sequence models
directly useful, easy to use to correct sentences	might not correspond to end task optimization
nice theoretical interpretation - matching distributions	$\log 0$ undefined
	can be cheated by predicting common tokens
	size of test set matters
	can be sensitive to low prob tokens/ sentences

Generalization

Number of Parameters

How many probabilities in each n -gram model

$$\mathcal{V} = \{\text{cat}, \text{crazy}, \text{mat}, \text{sat}\}$$

Question

How many unigram probabilities: $P(x)$ for $x \in \mathcal{V}$?

Number of Parameters

How many probabilities in each n -gram model

$$\mathcal{V} = \{\text{cat}, \text{crazy}, \text{mat}, \text{sat}\}$$

Question

How many bigram probabilities: $P(y|x)$ for $x, y \in \mathcal{V}$?

Number of Parameters

How many probabilities in each n -gram model

$$\mathcal{V} = \{\text{cat}, \text{crazy}, \text{mat}, \text{sat}\}$$

Question

How many trigram probabilities: $P(z|x, y)$ for $x, y, z \in \mathcal{V}$?

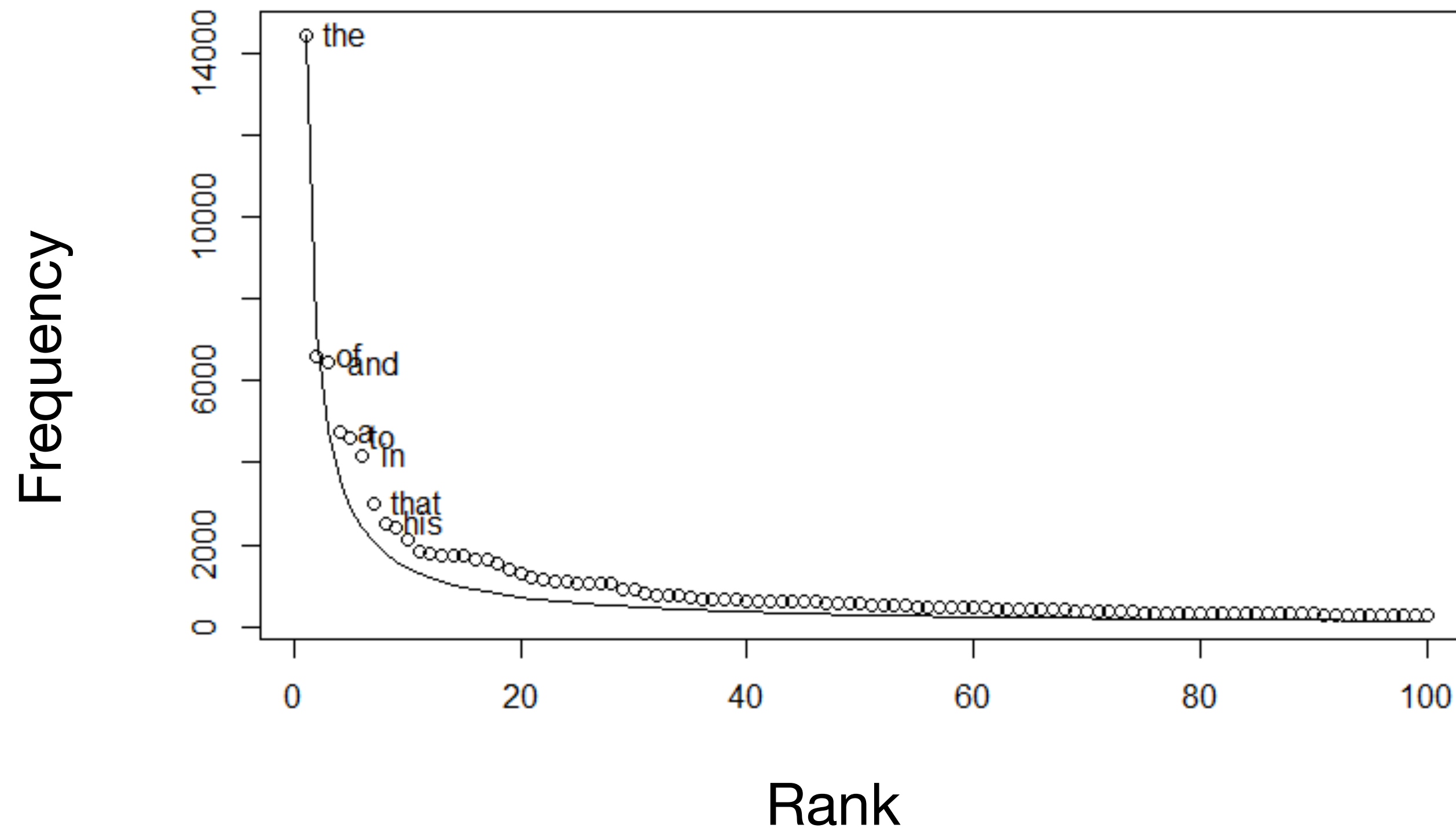
Number of parameters

- ▶ Assume $|\mathcal{V}| = 50,000$ (a realistic vocabulary size for English)
- ▶ What is the minimum size of training data in tokens?
 - ▶ If you wanted to observe all unigrams at least once.
 - ▶ If you wanted to observe all trigrams at least once.

Generalization of n-grams

- Not all n-grams will be observed in training data!
- There can be unknown words in the test set!
- Test corpus might have some that have zero probability under our model
- Training set: *Google news*
- Test set: *Shakespeare*
- $P(\text{affray} \mid \text{voice doth us}) = 0 \quad \Rightarrow \quad P(\text{test corpus}) = 0$

Sparsity in language



$$freq \propto \frac{1}{rank}$$

Zipf's Law

- Long tail of infrequent words
- Most finite-size corpora will have this problem.

Unknown words

- Typically assume closed vocabulary
- What about words not in the vocabulary?
 - Known as OOV (out-of-vocabulary) words.
 - Introduce **<UNK> token** to represent the unknown words
- If we never see these words in the training data, any sentence with these words will get a probability of 0!
- Can handle these unknown words by:
 - Estimate the probability of unknown word as: $P_{\text{unk}}(w) = \frac{1}{|V_{\text{all}}|}$ $V_{\text{all}} = V \cup \{< \text{UNK} >\}$
 - Or modify training data so rare words (words that appear $< T$ times) are treated as **<UNK>**

Smoothing n-gram Models

Smoothing

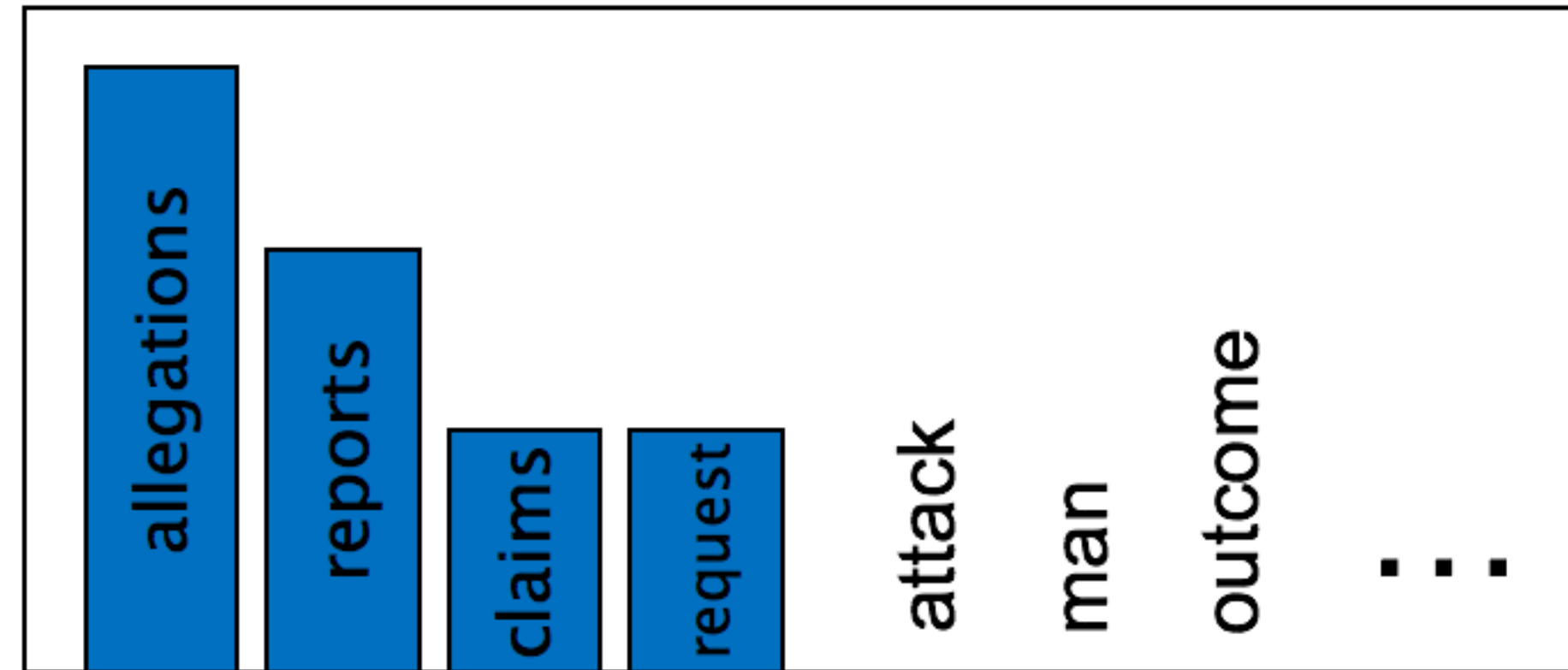
- **Smoothing** deals with events that have been observed zero or very few times
- Handle sparsity by making sure all **probabilities are non-zero** in our model
- **Additive**: Add a small amount to all probabilities
- **Discounting**: Redistribute probability mass from observed n-grams to unobserved ones
- **Interpolation**: Use a combination of different n-grams
- **Back-off**: Use lower order n-grams if higher ones are too sparse

Smoothing intuition

Taking from the rich and giving to the poor

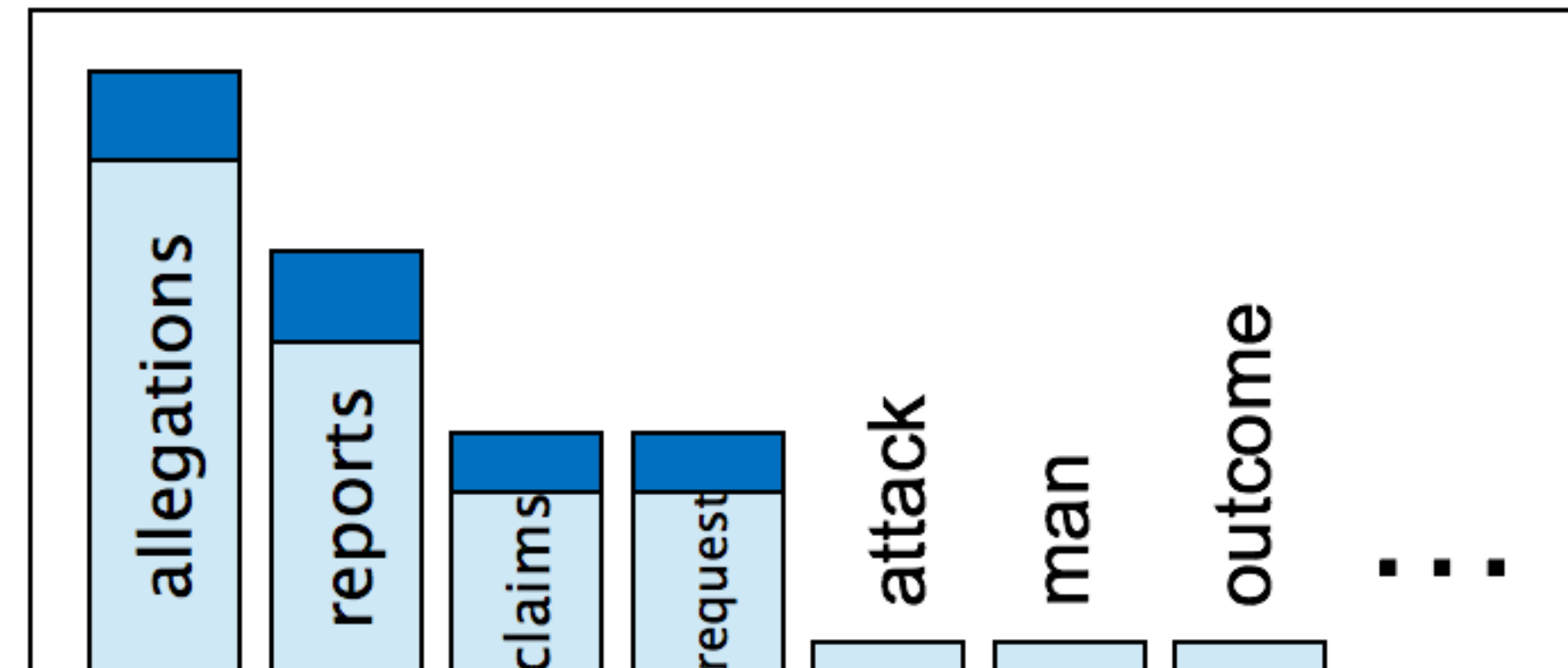
When we have sparse statistics:

$P(w \mid \text{denied the})$
3 allegations
2 reports
1 claims
1 request
7 total



Steal probability mass to generalize better

$P(w \mid \text{denied the})$
2.5 allegations
1.5 reports
0.5 claims
0.5 request
2 other
7 total



(Credits: Dan Klein)

Add-one (Laplace) smoothing

- Why add 1? 1 is an overestimate for unobserved events

- Max likelihood estimate for bigrams: $P_{\text{ML}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$

- Let $|V|$ be the number of words in our vocabulary. Assign count of 1 to unseen bigrams

- After smoothing:

$$P_{\text{Add1}}(w_i | w_{i-1}) = \frac{1 + C(w_{i-1}, w_i)}{|V| + C(w_{i-1})}$$

Additive smoothing

(Lidstone 1920, Jeffreys 1948)

- Why add 1? 1 is an overestimate for unobserved events

$$P_{\text{Add1}}(w_i | w_{i-1}) = \frac{1 + C(w_{i-1}, w_i)}{|V| + C(w_{i-1})}$$

- Additive smoothing ($0 < \delta \leq 1$):

$$P_{\text{Add}\delta}(w_i | w_{i-1}) = \frac{\delta + C(w_{i-1}, w_i)}{\delta \times |V| + C(w_{i-1})}$$

- Also known as add-alpha (the symbol α is used instead of δ)

Raw bigram counts (Berkeley restaurant corpus)

$$P_{\text{ML}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

- Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

(Credits: Dan Jurafsky)

Smoothed bigram counts

	i	want	to	eat	chinese	food	lunch	spend
i	6	828	1	10	1	1	1	3
want	3	1	609	2	7	7	6	2
to	3	1	5	687	3	1	7	212
eat	1	1	3	1	17	3	43	1
chinese	2	1	1	1	1	83	2	1
food	16	1	16	1	2	5	1	1
lunch	3	1	1	1	1	2	1	1
spend	2	1	2	1	1	1	1	1

Smoothed bigram probabilities

(Laplace Add-1 smoothing)

$$P_{\text{Add1}}(w_i | w_{i-1}) = \frac{1 + C(w_{i-1}, w_i)}{|V| + C(w_{i-1})}$$

	i	want	to	eat	chinese	food	lunch	spend
i	0.0015	0.21	0.00025	0.0025	0.00025	0.00025	0.00025	0.00075
want	0.0013	0.00042	0.26	0.00084	0.0029	0.0029	0.0025	0.00084
to	0.00078	0.00026	0.0013	0.18	0.00078	0.00026	0.0018	0.055
eat	0.00046	0.00046	0.0014	0.00046	0.0078	0.0014	0.02	0.00046
chinese	0.0012	0.00062	0.00062	0.00062	0.00062	0.052	0.0012	0.00062
food	0.0063	0.00039	0.0063	0.00039	0.00079	0.002	0.00039	0.00039
lunch	0.0017	0.00056	0.00056	0.00056	0.00056	0.0011	0.00056	0.00056
spend	0.0012	0.00058	0.0012	0.00058	0.00058	0.00058	0.00058	0.00058

(Credits: Dan Jurafsky)

The problem with Laplace smoothing

Too much discounted
from popular words!

Raw
counts

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Reconstituted
counts

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

$$c^*(w_{n-1}w_n) = \frac{[C(w_{n-1}w_n) + 1] \times C(w_{n-1})}{C(w_{n-1}) + V}$$

(Credits: Dan Jurafsky)

Linear Interpolation (Jelinek-Mercer Smoothing)

$$P_{\text{interp}}(w_i | w_{i-1}, w_{i-2}) = \lambda_1 P(w_i | w_{i-1}, w_{i-2}) \\ + \lambda_2 P(w_i | w_{i-1}) \\ + \lambda_3 P(w_i) \\ + \lambda_4 \frac{1}{|V|}$$
$$\sum_i \lambda_i = 1$$

- Use a combination of models to estimate probability
- Strong empirical performance

Jelinek and Mercer (1980)

Linear Interpolation (Jelinek-Mercer Smoothing)

- It's also possible to formulate the interpolation in a recursive manner:

$$P_{\text{JM}}(n\text{gram}) = \lambda_n P_{\text{ML}}(n\text{gram}) + (1 - \lambda_n) P_{\text{JM}}(n - 1\text{gram})$$

$$P_{\text{JM}}(w_i | w_{i-n+1}^{i-1}) = \lambda_n P_{\text{ML}}(w_i | w_{i-n+1}^{i-1}) + (1 - \lambda_n) P_{\text{JM}}(w_i | w_{i-n+2}^{i-1})$$

$$P_{\text{JM}}(w_i) = \lambda_1 P_{\text{ML}}(w_i) + (1 - \lambda_1) \frac{\delta}{|V|}$$

Brown et al (1992)

Linear Interpolation: Finding lambda

$$P_{\text{JM}}(\text{ngram}) = \lambda P_{\text{ML}}(\text{ngram}) + (1 - \lambda) P_{\text{JM}}(n - 1\text{gram})$$

- Interpolation parameters (λ) are hyper parameters. Tune them on the “held-out” set.



- Improved JM smoothing, a different λ for each w_i

$$P_{\text{JM}}(w_i | w_{i-1}) = \lambda(w_{i-1}) P_{\text{ML}}(w_i | w_{i-1}) + (1 - \lambda(w_{i-1})) P_{\text{JM}}(w_i)$$

Discounting

Bigram count in training	Bigram count in heldout set
0	.0000270
1	0.448
2	1.25
3	2.24
4	3.23
5	4.21
6	5.23
7	6.21
8	7.21
9	8.26

- Determine some “mass” to remove from probability estimates
- Redistribute mass among unseen n-grams
- Just choose an absolute value (D) to discount

more properly

$$\max(c(w_{i-1}, w_i) - D, 0)$$

α is set so the resulting probability values sums to one

Interpolated
absolute
discounting

$$P_{\text{absdis-i}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) - D}{C(w_{i-1})} + \alpha(w_{i-1}) P_{\text{absdis-i}}(w_i)$$

Very similar to
Interpolated
Knesser-Ney

With interpolation, can also be with “backoff” as we will see

Interpolated Knesser-Ney

Popular state of the art n-gram smoothing

Cleverer count (based on number of contexts)

Modified Knesser-Ney: different discounting values

$$P_{\text{KN-i}}(w_i | w_{i-1}) = \frac{\max(C_{\text{KN}}(w_{i-1}, w_i) - D)}{\sum_{w'} C_{\text{KN}}(w_{i-1}, w')} + \alpha(w_{i-1}) P_{\text{KN-i}}(w_i)$$

more properly

$\max(c(w_{i-1}, w_i) - D, 0)$

α is set so the resulting probability values sums to one

Interpolated absolute discounting

$$P_{\text{absdis-i}}(w_i | w_{i-1}) = \frac{C(w_{i-1}, w_i) - D}{C(w_{i-1})} + \alpha(w_{i-1}) P_{\text{absdis-i}}(w_i)$$

Very similar to Interpolated Knesser-Ney

With interpolation, can also be with “backoff” as we will see

Back-off

- Use **n-gram** if enough evidence, else back off to **(n-1)-gram**

$$P_{\text{bo}}(w_i | w_{i-n+1}^{i-1})$$
$$= \begin{cases} d_{w_{i-n+1}^i} \frac{C(w_{i-n+1}^i)}{C(w_{i-n+1}^{i-1})} & \text{if } C(w_{i-n+1}^i) > 0 \\ \alpha_{w_{i-n+1}^{i-1}} P_{\text{bo}}(w_i | w_{i-n+2}^{i-1}) & \text{otherwise} \end{cases}$$

- d = amount of discounting (Katz back-off, 1987)
- α = back-off weight

Backoff Smoothing with Discounting

- Absolute Discounting with backoff (Ney, Essen, Knessler)

$$P_{\text{absdis-bo}}(w_i | w_{i-1}) = \begin{cases} \frac{C(w_{i-1} w_i) - D}{C(w_{i-1})} & \text{if } C(w_{i-1} w_i) > 0 \\ \alpha(w_{i-1}) P_{\text{absdis-bo}}(w_i) & \text{otherwise} \end{cases}$$

- Where $\alpha(w_{i-1})$ is chosen to ensure that $P_{\text{abs}}(w_i | w_{i-1})$ is a proper probability

Similar to
Backoff
Knessler-Ney,
1994

$$\alpha(w_{i-1}) = 1 - \sum_{w_i} \frac{C(w_{i-1} w_i) - D}{C(w_i)}$$

Different value of α for
each context word w_{i-1}

Backoff Smoothing with Discounting

- Let $D = 0.5$

- Missing probability mass:

$$\alpha(w_{i-1}) = 1 - \sum_{w_i} \frac{C(w_{i-1}w_i) - D}{C(w_i)}$$

$$\alpha(\text{the}) = 10 \times 0.5/48 = 5/48$$

- Divide this mass between words w for which the counts: $C(\text{the}, w) = 0$

x	$c(x)$	$c(x) - D$	$\frac{c(x) - D}{c(\text{the})}$
the	48		
the,dog	15	14.5	14.5/48
the,woman	11	10.5	10.5/48
the,man	10	9.5	9.5/48
the,park	5	4.5	4.5/48
the,job	2	1.5	1.5/48
the,telescope	1	0.5	0.5/48
the>manual	1	0.5	0.5/48
the,afternoon	1	0.5	0.5/48
the,country	1	0.5	0.5/48
the,street	1	0.5	0.5/48
TOTAL			0.8958
the,UNK	0		0.1042

Web-scale N-grams Smoothing

Keeping track of everything gets complicated

Not even a proper distribution!

- “Stupid backoff” (Brants et al, 2007)

$$S(w_i | w_{i-n+1}^{i-1}) = \begin{cases} \frac{C(w_{i-n+1}^i)}{C(w_{i-n+1}^{i-1})} & \text{if } C(w_{i-n+1}^i) > 0 \\ 0.4 S(w_i | w_{i-n+2}^{i-1}) & \text{otherwise} \end{cases}$$

S = Score

$$S(w_i) = \frac{C(w_i)}{|V|}$$

Other challenges

- Efficient storage
- Efficient lookup

<https://www.aclweb.org/anthology/D07-1090.pdf>

Beyond n-grams

Other types of language models

- Discriminative models:
 - train n-gram probabilities to directly maximize performance on end task (e.g. as feature weights)
- Parsing-based models
 - handle syntactic/grammatical dependencies
- Topic models
- Neural Language Models

Typical values of Perplexity

From 'One Billion Word Benchmark for Measuring Progress in Statistical Language Modeling' by Chelba+ (Google) 2013

Model	Num. Params [billions]	Training Time		Perplexity
		[hours]	[CPUs]	
Interpolated KN 5-gram, 1.1B n-grams (KN)	1.76	3	100	67.6
Katz 5-gram, 1.1B n-grams	1.74	2	100	79.9
Stupid Backoff 5-gram (SBO)	1.13	0.4	200	87.9
Interpolated KN 5-gram, 15M n-grams	0.03	3	100	243.2
Katz 5-gram, 15M n-grams	0.03	2	100	127.5
Binary MaxEnt 5-gram (n-gram features)	1.13	1	5000	115.4
Binary MaxEnt 5-gram (n-gram + skip-1 features)	1.8	1.25	5000	107.1
Hierarchical Softmax MaxEnt 4-gram (HME)	6	3	1	101.3
Recurrent NN-256 + MaxEnt 9-gram	20	60	24	58.3
Recurrent NN-512 + MaxEnt 9-gram	20	120	24	54.5
Recurrent NN-1024 + MaxEnt 9-gram	20	240	24	51.3

Transformer-XL Large [Dai et al 2018]

0.8

21.8

1B Word Benchmark test set

Where are we now?

Neural models with lots of data!

300 Billion tokens

Training data: mix of web + books + Wikipedia

Model Name	n_{params}
GPT-3 Small	125M
GPT-3 Medium	350M
GPT-3 Large	760M
GPT-3 XL	1.3B
GPT-3 2.7B	2.7B
GPT-3 6.7B	6.7B
GPT-3 13B	13.0B
GPT-3 175B or “GPT-3”	175.0B

Dataset	1B Word Benchmark test set	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)		410 billion	60%	0.44
WebText2		19 billion	22%	2.9
Books1		12 billion	8%	1.9
Books2		55 billion	8%	0.43
Wikipedia		3 billion	3%	3.4

Open AI's GPT 3

Language Models are Few-Shot Learners
(Brown et al, 2020)

<https://arxiv.org/pdf/2005.14165.pdf>

Setting	PTB
SOTA (Zero-Shot)	35.8 ^a
GPT-3 Zero-Shot	20.5

Summary: Estimating language models

- ▶ Predict probability of sequence of words
- ▶ Need to handle **data sparsity**
use Markov assumption and smoothing
Independence assumptions
Reallocate probability mass
Ensure proper probability
- ▶ Later: Neural language models

Use Chain rule and approximate using a neural network

$$p(w_1, \dots, w_n) \approx \prod_t p(w_{t+1} \mid \underbrace{\phi(w_1, \dots, w_t)}_{\text{capture history with vector } s(t)})$$